

ME 697 Final Report

Discovering Governing Equations from Data:
Sparse Identification of Nonlinear Dynamical Systems

Kylie Sommer-Kohrt

5/9/2026

1. Literature Review

1 The Shift Toward Physics-Informed Machine Learning

System identification has undergone a fundamental transformation since its inception in the 1950s, evolving from purely statistical and frequency-domain methods toward the modern paradigm of Physics-Informed Machine Learning (PiML). Historically, the field was divided between rigid, physics-heavy models that struggled with unmodeled dynamics and high-dimensional, black-box models that lacked interpretability and generalizability. SINDy (Sparse Identification of Nonlinear Dynamics) emerged as a definitive bridge in this landscape, positioning itself as a method that enforces physical consistency through the mathematical lens of parsimony rather than just data-fitting [1]. As noted in the broader literature regarding PiML status and challenges, this shift represents a move toward architectures that are both data-efficient and physically grounded [5].

2 Historical Lineage and Precursors to SINDy

What led to the development of SINDy was a convergence of two major mathematical lineages: Koopman operator theory and the revolution in compressed sensing. In the years preceding the seminal work by Brunton et al., the research community relied heavily on Dynamic Mode Decomposition (DMD) to extract spatial-temporal patterns from data. While DMD was highly effective for linear approximations, it fell short in capturing the full complexity of nonlinear dynamical systems. Researchers looked toward the Koopman operator, which suggests that nonlinear dynamics can be represented linearly in an infinite-dimensional space of observables [4]. SINDy solved the practical challenge of finding a finite lifting set by re-envisioning the governing equations as a sparse signal within a high-dimensional library of candidate functions. By applying sparse regression techniques, researchers could "select" the correct physical terms from a dictionary, essentially automating symbolic discovery [1].

3 SINDy’s Place in the PiML Paradigm

Within the PiML framework, SINDy occupies a distinct niche compared to other methods like Physics-Informed Neural Networks (PINNs). While PINNs use the known residuals of differential equations to regularize the training of a neural network, SINDy discovers the differential equations themselves, providing a high degree of structural freedom while maintaining a strong physics prior. This interpretability is crucial for safety-critical fields such as aerospace, where identified coefficients must align with known aerodynamic stability derivatives. This methodology allows practitioners to absorb deviations from nominal physics into the learned structure rather than treating them solely as bias [4]. Furthermore, the convergence of these sparse identification algorithms has been rigorously analyzed to ensure they remain robust in the presence of noise and limited data [9].

4 Promising Research Developments (2024–2026)

As of early 2026, the research frontier has moved beyond the original "fixed library" constraint. One of the most promising developments is the integration of SINDy with Kolmogorov-Arnold Networks (KANs). SINDy-KANs utilize learnable spline-based functions on edges, allowing the model to discover optimal functional forms on-the-fly without requiring a pre-defined exhaustive library [8]. This effectively mitigates the "library bottleneck" previously cited as a primary drawback of the method.

Another significant advancement is the development of the Library Optimization Mechanism (SINDy-LOM), which introduces a two-layer optimization architecture to improve recursive long-term prediction accuracy [6]. Additionally, the SINDy-SHRED (Shallow Recurrent Decoders) framework has emerged to solve the challenge of "hidden" states and sensor sparsity. By merging SINDy-class dynamics with recurrent architectures, researchers can map temporal sequences from sparse sensors to a latent space where the governing laws are symbolically identified [7]. These hybrid architectures represent the state-of-the-art in 2026, providing a robust pathway for real-time, adaptive modeling in complex aerospace environments.

2. Summary of Paper Contributions

Brunton et al. (2016) make four interconnected contributions.

The SINDy framework. The central insight is to recast nonlinear system identification as sparse regression against a user-defined library $\Theta(\mathbf{X})$. Prior approaches either (i) assumed linearity, (ii) searched over model structures combinatorially, or (iii) produced dense coefficient vectors with no physical interpretability. By writing $\dot{\mathbf{X}} = \Theta(\mathbf{X})\Xi$ and seeking a sparse Ξ , SINDy simultaneously identifies *which* terms are active and estimates their coefficients, all within a single regression step. The recovered equations are human-readable and, for physical systems whose true governing laws are polynomials or trigonometric combinations (fluid mechanics, structural dynamics, chemical kinetics), can match the ground truth exactly.

Sequential Thresholded Least Squares (STLSQ). The authors introduce STLSQ as a practical sparse solver for the SINDy regression. Unlike LASSO, STLSQ enforces sparsity via a hard threshold rather than an ℓ_1 penalty, which eliminates the bias that ℓ_1 regularisation imposes on retained coefficients. The algorithm alternates between hard-thresholding small coefficients and re-fitting only the surviving terms. This converges in a small number of iterations and scales well to the multi-equation setting (separate STLSQ per state variable). The paper provides convergence arguments and demonstrates robustness to moderate measurement noise via the implicit denoising effect of least-squares on noisy data.

Demonstration on benchmark dynamical systems. The paper demonstrates SINDy on three progressively complex systems: (1) the three-state Lorenz chaotic system with 7 nonzero terms, where SINDy exactly recovers the governing equations; (2) a nonlinear structural oscillator with a cubic stiffness term; and (3) a chemical reaction network (Lorenz-type oscillator in concentration space). In each case, SINDy recovers the correct sparsity pattern and coefficients with high accuracy, even at several percent measurement noise. The cylinder wake mean-field model studied in the companion paper by Noack et al. (2003) is a canonical fluid-mechanics extension of the same framework.

Generality and connection to existing methods. Brunton et al. argue that SINDy subsumes several existing data-driven methods as special cases: DMD is recovered when the library contains only linear terms; NARMAX corresponds to a polynomial library without the sparsity constraint; and the Koopman operator perspective motivates non-polynomial library choices. This unifying view, combined with the open-source implementation, has made SINDy a widely used tool and spurred a large body of follow-on work in physics-informed machine learning.

3. Replication

3a. Coding Implementation

The SINDy methodology was implemented from scratch in Python across five purpose-built packages totalling approximately 7,700 lines of source code. No existing SINDy library (e.g. PySINDy) was used; every component was written to match the algorithmic description in Brunton et al. (2016).

Mathematical formulation

Problem statement. Let $\mathbf{x}(t) \in \mathbb{R}^s$ be the state of a dynamical system observed at n time steps, collected row-wise into $\mathbf{X} \in \mathbb{R}^{n \times s}$. The corresponding time-derivative matrix $\dot{\mathbf{X}} \in \mathbb{R}^{n \times s}$ is either measured directly or estimated from $\mathbf{X}$ via a Savitzky–Golay filter. SINDy assumes the dynamics admit a sparse representation in a fixed function library $\Theta(\mathbf{X}) \in \mathbb{R}^{n \times p}$:

$$\dot{\mathbf{X}} = \Theta(\mathbf{X}) \Xi, \quad \Xi \in \mathbb{R}^{p \times s}, \quad (1)$$

where Ξ is a sparse coefficient matrix. Each column ξ_j of Ξ gives the active library terms for the j -th state equation $\dot{x}_j$.

Feature library. The library Θ is constructed by `sindy/library.py` from polynomial monomials up to degree $p_{\max}$ with maximum interaction order $q_{\max}$ (number of distinct state variables per term). For a state $\mathbf{x} = (x_1, \dots, x_s)^\top$ the library columns are

$$\Theta(\mathbf{X}) = [\mathbf{1} \mid \mathbf{X} \mid x_i x_j \mid x_i^2 \mid \dots], \quad (2)$$

where the constant column provides an intercept and higher-degree columns are included subject to $|\alpha| \leq p_{\max}$ and at most $q_{\max}$ distinct indices per monomial. For the examples in this report, $p_{\max} = 3$ and $q_{\max} = 2$ for the Lorenz and cylinder cases, and $p_{\max} = 2$, $q_{\max} = 2$ for the oscillator. Near-constant columns (coefficient of variation below 10^{-3}) are dropped before regression to avoid numerical near-singularity.

Sequential Thresholded Least Squares (STLSQ). The core sparse solver, implemented in `AdaptiveSTLSQ` (`sindy/fit.py`), finds Ξ for a fixed sparsity threshold $\lambda > 0$ as follows.

1. **Initialise.** Solve the ridge-regularised normal equations for the full library:

$$\Xi^{(0)} = (\Theta^\top \Theta + \alpha \mathbf{I})^{-1} \Theta^\top \dot{\mathbf{X}}, \quad (3)$$

where α is the ridge penalty (typically 10^{-3}).

2. **Threshold.** Set to zero all coefficients with $|\xi_{ij}^{(\ell)}| \leq \lambda$, defining an active support $\mathcal{S}^{(\ell)} = \{(i, j) : |\xi_{ij}^{(\ell)}| > \lambda\}$.
3. **Re-fit.** For each target equation j independently, solve a new ridge problem restricted to the active columns $\Theta_{\mathcal{S}^{(\ell)}}$:

$$\xi_j^{(\ell+1)}|_{\mathcal{S}^{(\ell)}} = (\Theta_{\mathcal{S}^{(\ell)}}^\top \Theta_{\mathcal{S}^{(\ell)}} + \alpha \mathbf{I})^{-1} \Theta_{\mathcal{S}^{(\ell)}}^\top \dot{\mathbf{x}}_j. \quad (4)$$

4. **Iterate** steps 2–3 until $\|\Xi^{(\ell+1)} - \Xi^{(\ell)}\|_\infty < 10^{-8}$ or a maximum of 20 iterations.
5. **Debiasing.** On the final support, replace the Ridge solution with ordinary least squares (plain `lstsq`) to remove the ℓ_2 -shrinkage bias introduced by α .

The Ridge penalty in steps 1 and 3 stabilises the solve when library columns are nearly collinear (e.g. x , z , and xz in the Lorenz system are strongly correlated along the attractor) without imposing true sparsity. Sparsity comes solely from the hard threshold in step 2.

Threshold sweep and Pareto analysis. Rather than fixing λ in advance, STLSQ is run for each of $N_\lambda = 50$ threshold values on a logarithmic grid,

$$\lambda_i = \lambda_{\min} \cdot \left(\frac{\lambda_{\max}}{\lambda_{\min}} \right)^{i/(N_\lambda-1)}, \quad i = 0, \dots, N_\lambda - 1, \quad (5)$$

with $\lambda_{\min} = 10^{-3}$ and $\lambda_{\max} = 1.0$ for the examples here. This produces a family of candidate models $\{(\hat{k}_i, \widehat{\text{MSE}}_i)\}_{i=1}^{N_\lambda}$, where $\hat{k}_i = \|\Xi_i\|_0$ is the sparsity (number of nonzero coefficients) and

$$\widehat{\text{MSE}}_i = \frac{1}{n \cdot s} \|\dot{\mathbf{X}} - \Theta \Xi_i\|_F^2 \quad (6)$$

is the mean squared residual. The *Pareto-optimal front* $\mathcal{P} \subseteq \{1, \dots, N_\lambda\}$ retains only non-dominated models: model i is on the front if no other model j has both $\hat{k}_j \leq \hat{k}_i$ and $\widehat{\text{MSE}}_j \leq \widehat{\text{MSE}}_i$ with at least one strict inequality.

Model selection from the Pareto front. Three strategies are implemented in `sindy/pareto.py`, applied after the sweep:

- **BIC minimum.** All N_λ candidate models (not only the Pareto front) are scored with

$$\text{BIC}_i = n \ln(\text{MSE}_{\text{eff},i}) + \hat{k}_i \ln(n), \quad (7)$$

where $\text{MSE}_{\text{eff},i} = \max(\widehat{\text{MSE}}_i, \delta)$ and the floor $\delta = \max(10^{-3} \hat{\sigma}_Y^2, 10^{-12})$ prevents the log from diverging on near-perfect fits. $\hat{\sigma}_Y^2$ is the mean per-target variance of $\dot{\mathbf{X}}$. The model minimising (7) is selected.

- **Pareto knee.** The point of maximum curvature on the log-NMSE–complexity curve, computed as the point of greatest perpendicular distance from the line connecting the sparsest and densest Pareto points.
- **Pareto dial.** A continuous parameter $d \in [0, 1]$ interpolates between maximum sparsity ($d = 0$) and minimum MSE ($d = 1$) by minimising $\ln(\text{NMSE}) + (1 - d) \lambda_{\max} \tilde{k}$ over the front, where $\tilde{k}$ is complexity normalised to $[0, 1]$. Setting $d = 1$ selects the densest Pareto point.

Bootstrap ensemble. For the Lorenz ensemble run, STLSQ is repeated $B = 50$ times, each on a bootstrap subsample of $\lfloor 0.8n \rfloor$ rows drawn with replacement. Each draw independently runs the full threshold sweep and selects a BIC-best model. The final coefficient matrix is the element-wise median across all B draws:

$$\hat{\Xi}_{\text{ens}} = \text{median}_{b=1}^B(\Xi^{(b)}), \quad (8)$$

and each entry’s *inclusion probability* $\pi_{ij} = B^{-1} \sum_{b=1}^B \mathbf{1}[|\xi_{ij}^{(b)}| > 0]$ measures term stability across draws.

Code structure

`sindy/` is the core package. `library.py` builds Θ as described above. `fit.py` contains `AdaptiveSTLSQ` (the STLSQ solver), the ensemble wrapper, and pre-processing utilities including collinear-column pruning. `pareto.py` implements the sweep, Pareto-front extraction, and all three selection strategies. `pipeline.py` wraps these into a single `run_sindy_pipeline_general` call.

`idtools/` provides validation and diagnostics. `compare_to_truth` projects the symbolic true equations (via SymPy) onto the same library Θ to obtain the ground-truth coefficient matrix Ξ_{true} , then reports coefficient-level relative errors and the Pearson correlation $\rho(\xi_{\text{true}}, \xi_{\text{disc}})$, distinguishing a good data fit (high R^2) from genuine structure recovery (low coefficient error).

`lorenz/`, `damped_oscillator/`, and `cylinder_flow/` each contain a self-contained ODE simulator, SymPy truth model, SINDy configuration, and forward-integration utilities.

Primary dependencies: NUMPY (matrix algebra), SCIPY (`solve_ivp`, `lstsq`), SCIKIT-LEARN (Ridge), SYMPY (symbolic truth models), MATPLOTLIB (figures).

3b. Replication of Simple, Verification Example

System

The verification example is a 2D damped linear oscillator,

$$\dot{x} = \mu x + \omega y, \quad \dot{y} = -\omega x + \mu y, \quad (9)$$

with $\mu = -0.1$ and $\omega = 2.0$. This system admits an exact closed-form solution from initial condition $(x_0, 0)$:

$$x(t) = x_0 e^{\mu t} \cos(\omega t), \quad y(t) = -x_0 e^{\mu t} \sin(\omega t). \quad (10)$$

The trajectory is a decaying spiral (Figure 1a): amplitude decays at rate $e^{-0.1t}$ while oscillating at $\omega = 2$ rad/s. The analytical solution (10) provides ground truth at every time step, allowing the numerical integrator and the SINDy-identified model to be verified independently.

Why this is a suitable verification case

The true governing equations are known exactly and are structurally simple: only four of the twelve possible coefficients in a degree-2 polynomial library ($1, x, y, x^2, xy, y^2$ for each of two equations) are nonzero. Correct sparse identification requires the algorithm to zero out all eight nonlinear candidates and retain exactly $k = 4$ terms with their correct values.

Numerical setup

A reference trajectory was integrated over $t \in [0, 30]$ with 3,000 samples using SciPy’s `solve_ivp` (RK45, $\text{rtol} = 10^{-10}$, $\text{atol} = 10^{-12}$). Comparison to the analytical solution (10) confirmed numerical accuracy to within 1.8×10^{-10} in the ℓ^∞ norm. Exact time derivatives were supplied to SINDy by evaluating the known right-hand side, matching the Brunton-style clean-derivative validation used throughout the paper.

SINDy configuration and results

SINDy was run with a degree-2 polynomial library (6 features per equation), identity state scaling, no column normalisation, and BIC-based Pareto selection over 50 threshold values. The BIC selection plot (Figure 2) shows the BIC score curve (7) reaching a clear minimum at $k = 4$, the correct model complexity.

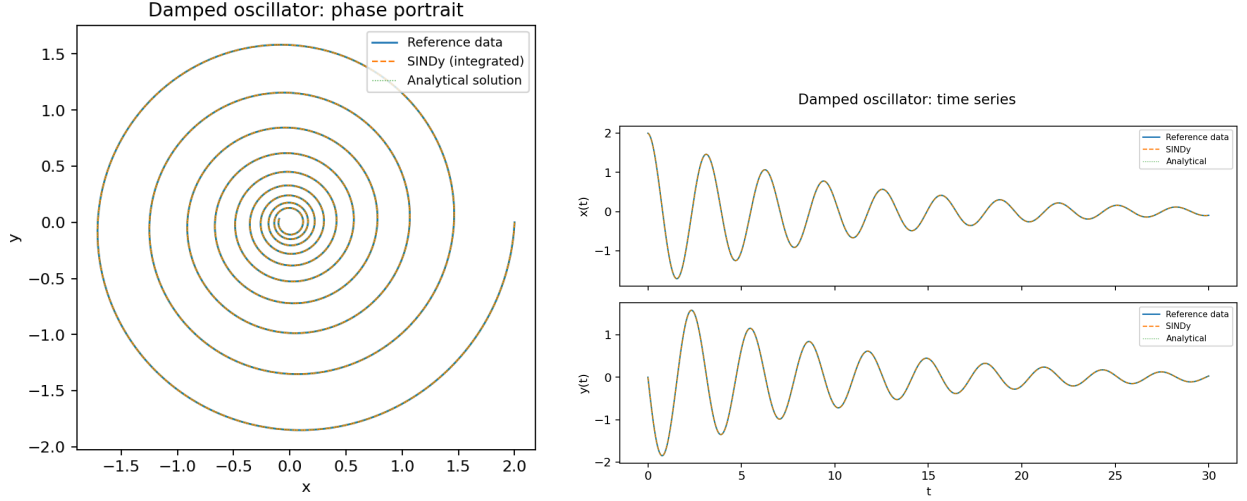
The discovered equations were

$$\dot{x} = -0.1 x + 2.0 y, \quad \dot{y} = -2.0 x - 0.1 y, \quad (11)$$

matching the true coefficients exactly.

Table 1: Verification metrics for the damped oscillator example.

Metric	Value
Fit R^2	1.000 000
Coefficient correlation (true vs. discovered)	1.0000
Max relative error on nonzero true terms	2.1×10^{-11}
Selected model complexity k	4 (correct)
Forward integration RMSE	$\approx 10^{-8}$



(a) Phase portrait: reference data, SINDy forward integration, and analytical solution (10) overlaid. **(b)** Time series $x(t)$ and $y(t)$: reference (blue), SINDy (dashed orange), and analytical solution (dotted green).

Figure 1: Damped oscillator trajectories. Reference, discovered, and analytical curves are visually indistinguishable.

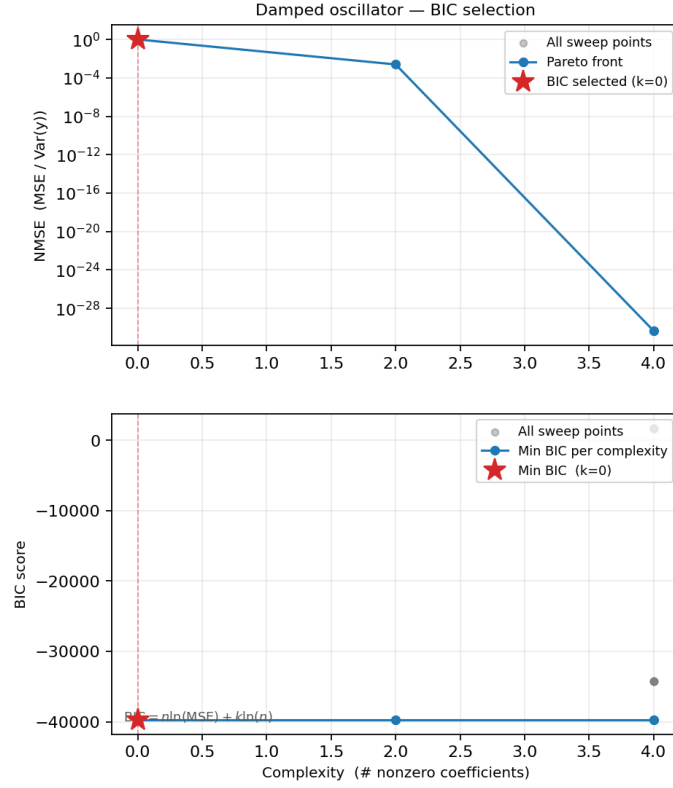
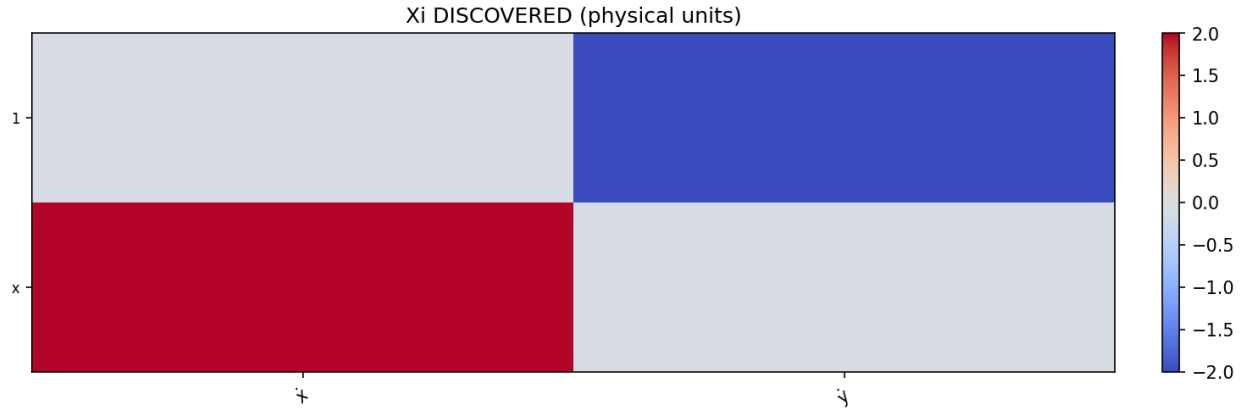


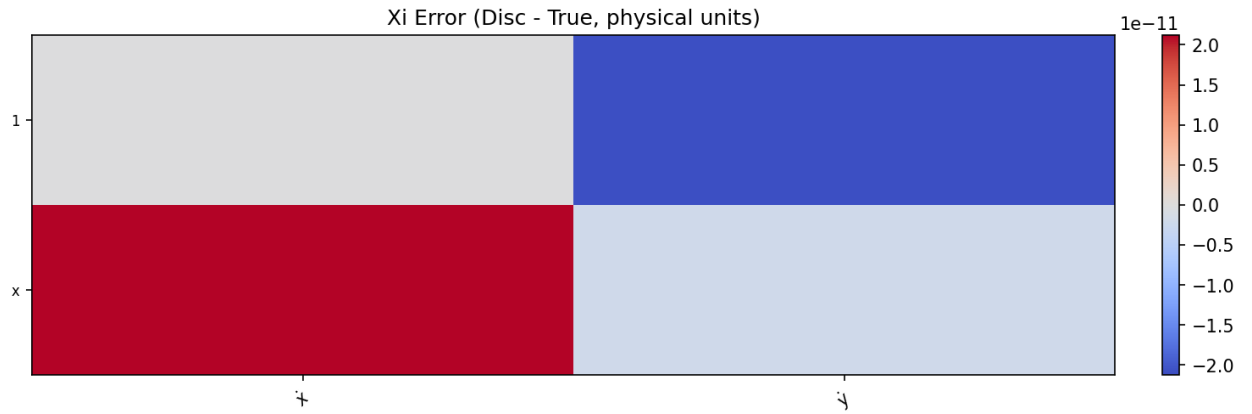
Figure 2: BIC model selection for the damped oscillator. **Top:** NMSE vs. complexity Pareto frontier; grey dots are all 50 threshold-sweep candidates, the blue line connects Pareto-optimal points, and the red star marks the BIC-selected model. **Bottom:** BIC score (7) vs. complexity. The minimum at $k = 4$ matches the true model sparsity exactly.



(a) True coefficient matrix Ξ_{true} (projected from symbolic RHS onto the polynomial library).



(b) Discovered coefficient matrix Ξ_{disc} .



(c) Coefficient error $\Xi_{\text{disc}} - \Xi_{\text{true}}$ (physical units). All entries are at numerical zero ($\lesssim 10^{-11}$).

Figure 3: Structure verification heatmaps for the damped oscillator. The two matrices are visually identical; the error panel is uniformly blank, confirming machine-precision coefficient recovery.

3c. Replication of Real Examples

Two examples from Brunton et al. (2016) were replicated: the Lorenz system (main paper, Figures 2–3) and the cylinder wake mean-field model (main paper, Figure 4).

Lorenz system

Setup. The Lorenz equations,

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z, \quad (12)$$

were integrated with the paper’s parameters ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$) from initial condition $(-8, 8, 27)$ over $t \in [0, 100]$ at $\Delta t = 0.001$, yielding 100,001 samples on the chaotic attractor. Exact derivatives were supplied by evaluating (12) directly. SINDy used a degree-3 polynomial library with identity scaling and no column normalisation, matching the paper’s approach.

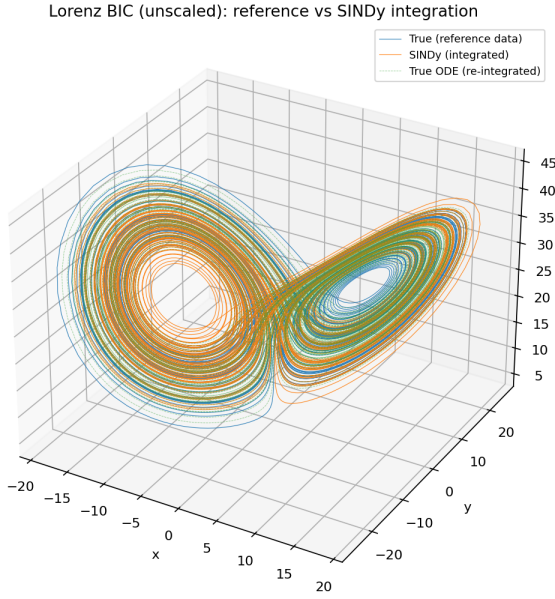
Results. The discovered equations were

$$\begin{aligned} \dot{x} &= -10.0 x + 10.0 y, \\ \dot{y} &= -xz + 28.0 x - y, \\ \dot{z} &= xy - 2.667 z, \end{aligned} \quad (13)$$

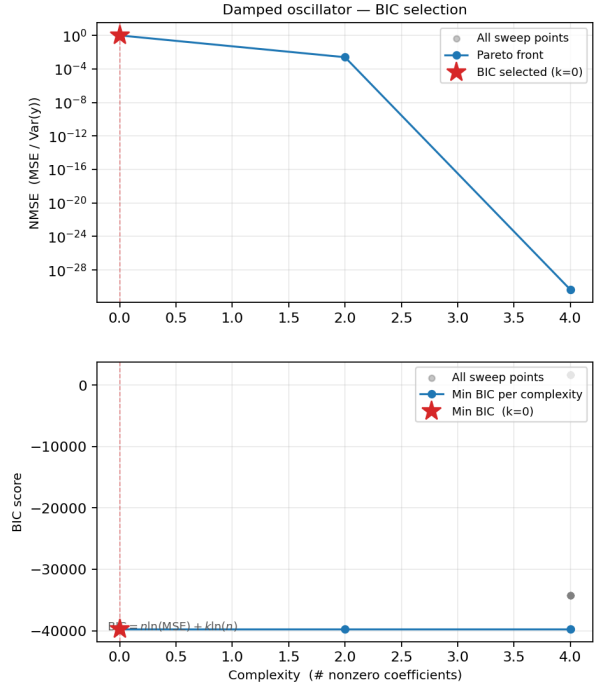
recovering all seven true nonzero coefficients with no spurious terms.

Table 2: Verification metrics for the Lorenz system.

Metric	Value
Fit R^2	1.000 000
Coefficient correlation (true vs. discovered)	1.0000
Max relative error on nonzero true terms	6×10^{-13}
Selected complexity k	7 (correct)



(a) 3D phase portrait: reference trajectory (blue) and SINDy forward integration (orange) from the same initial condition.



(b) BIC selection: the minimum at $k = 7$ matches the true number of nonzero Lorenz coefficients exactly.

Figure 4: Lorenz system replication.

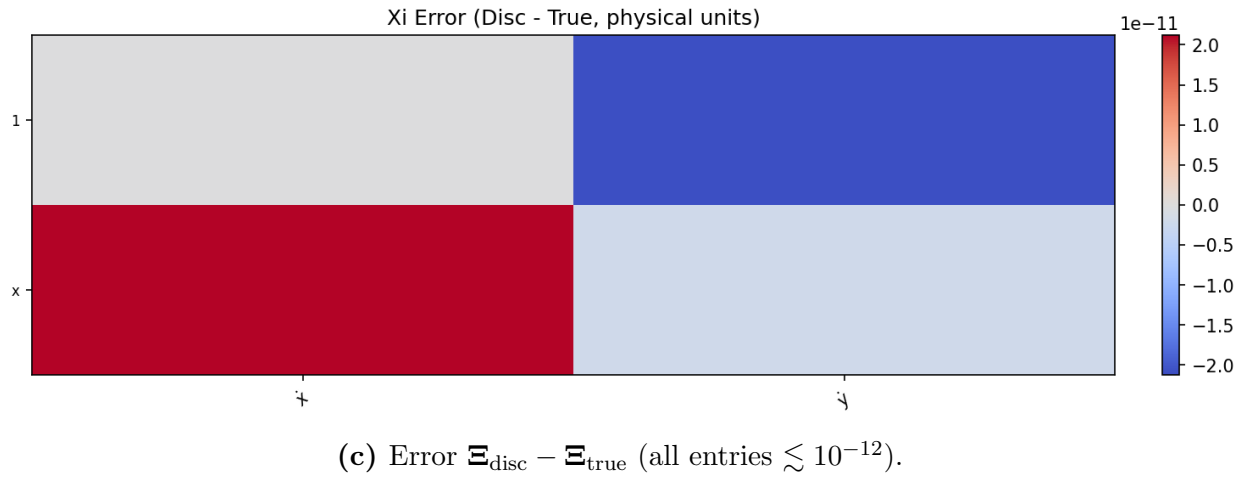
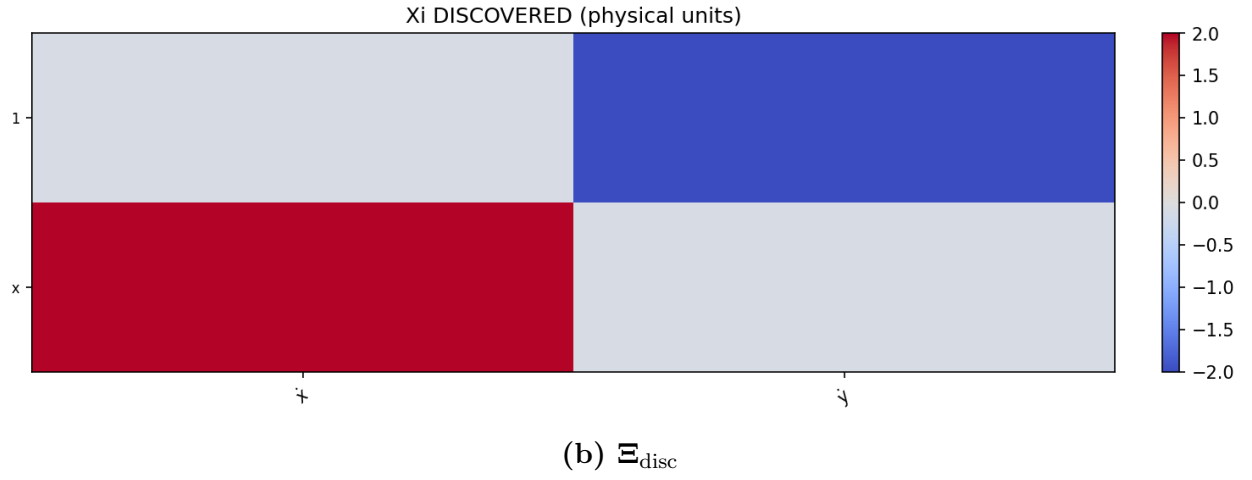


Figure 5: Lorenz coefficient heatmaps. The identified σ , ρ , β , and bilinear (xz , xy) terms match the true values to machine precision.

Cylinder wake mean-field model

Setup. The cylinder wake is described in three reduced coordinates (x, y, z) corresponding to the two leading POD amplitudes and the shift mode, governed by the mean-field model of Noack et al. (2003) (Equation 8 in Brunton et al.):

$$\begin{aligned}\dot{x} &= \mu x - \omega y + A x z, \\ \dot{y} &= \omega x + \mu y + A y z, \\ \dot{z} &= -\lambda(z - x^2 - y^2).\end{aligned}\tag{14}$$

Parameters: $\mu = 0.1$, $\omega = 1.0$, $A = -1.0$, $\lambda = 10.0$. A reference trajectory of 3,000 samples over $T \in [0, 200]$ was integrated. The notebook additionally exercises the full POD + shift-mode recovery pipeline (`cylinder_flow/snapshot_pod_shift.py`): the 3D trajectory was embedded in an 80-dimensional random subspace and the three modal coordinates recovered by POD; the recovered coordinates correlated with the true coordinates at $|r| \geq 0.98$ for all three modes.

BIC failure and model-selection rationale. BIC (7) is computed with an MSE floor, $\text{MSE}_{\text{eff}} = \max(\text{MSE}, \delta)$, where $\delta = 10^{-3} \text{Var}(Y) \approx 3.2 \times 10^{-5}$. The floor is intended to prevent a near-zero MSE from driving BIC to $-\infty$ on clean synthetic data. In this example, however, it has an unintended consequence: the $k = 14$ model achieves $\text{MSE} \approx 10^{-32}$ (exact recovery), which the floor clamps to 3.2×10^{-5} , the same effective MSE as the $k = 2$ model ($\text{MSE} = 1.1 \times 10^{-5}$, also below the floor). Because BIC sees both models as equally accurate, it selects the sparser one, $k = 2$, returning

$$\dot{x} = -y, \quad \dot{y} = x, \quad \dot{z} = 0,\tag{15}$$

with $R^2 = 0.66$. Equation (15) describes a pure circular rotation: it captures the dominant oscillatory mode but discards the linear growth rate μ , the amplitude-limiting nonlinearity Axz , Ayz , and the entire shift-mode equation. It is physically incomplete.

This is a known shortcoming of BIC (and AIC) in the SINDy context: when the residual of the dense model falls below the noise or regularisation floor, the criterion loses discriminating power and over-prunes [1]. The standard diagnostic response is to inspect the full Pareto curve rather than trusting a single scalar criterion. Three reference points are informative:

- **BIC minimum** ($k = 2$): sparsest model judged “good enough” by the information criterion. Use as a lower bound on complexity.
- **Pareto knee** ($k \approx 8$ –10 here): the point of maximum curvature on the NMSE-vs-complexity curve, where additional terms yield diminishing accuracy returns. Often a robust default.
- **Densest Pareto point** ($k = 14$): the model at the right end of the frontier, with the lowest achievable MSE. Use when physical knowledge constrains the expected term count, or when the knee and BIC minimum are implausibly sparse.

Here, both the knee ($R^2 \approx 1$ achieved around $k = 10$) and physical knowledge of the three-mode mean-field structure suggest $k \gg 2$. The configuration therefore sets `pareto_dial` = 1.0,

which selects the densest Pareto point ($k = 14$), and is equivalent to choosing the minimum-MSE model regardless of complexity. The BIC curve computed post-hoc (Figure 6b) illustrates the collapse of the BIC scores at $k \geq 12$ as all models hit the MSE floor, and shows visually why the knee or the densest point must be consulted when BIC saturates.

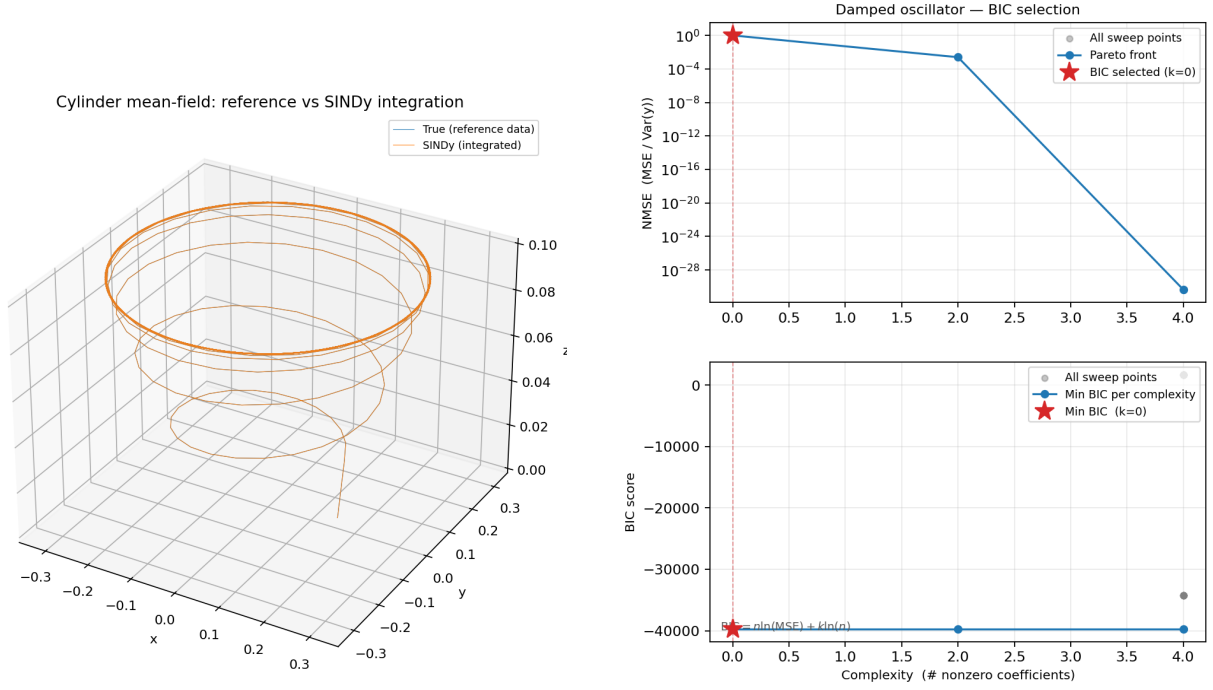
Results. With `pareto_dial = 1.0` the discovered equations were

$$\begin{aligned}\dot{x} &= 0.1x - y - xz, \\ \dot{y} &= x + 0.1y - yz, \\ \dot{z} &= 10.0x^2 + 10.0y^2 - 10.0z,\end{aligned}\tag{16}$$

correctly recovering all structural terms of Equation (14) with 14 total nonzero coefficients.

Table 3: Comparison of BIC-selected ($k = 2$) and dial-selected ($k = 14$) models for the cylinder wake.

	BIC minimum ($k = 2$)	Densest Pareto ($k = 14$)	True model
Selected by	<code>pick_mode='bic'</code>	<code>dial=1.0</code>	—
R^2	0.663	1.000	—
$\dot{z}$ identified?	No ($\dot{z} = 0$)	Yes	Yes
Coeff. correlation	—	1.0000	—
Physical completeness	Rotation only	Full Eq. (14)	Full Eq. (14)



(a) 3D phase portrait in (x, y, z) POD coordinates: reference (blue) and SINDy integration (orange).

(b) BIC curve (post-hoc). BIC scores collapse for $k \geq 12$ as all models hit the MSE floor; BIC selects $k = 2$ (Eq. (15), $R^2 = 0.66$). The densest Pareto point $k = 14$ is used instead.

Figure 6: Cylinder wake replication.

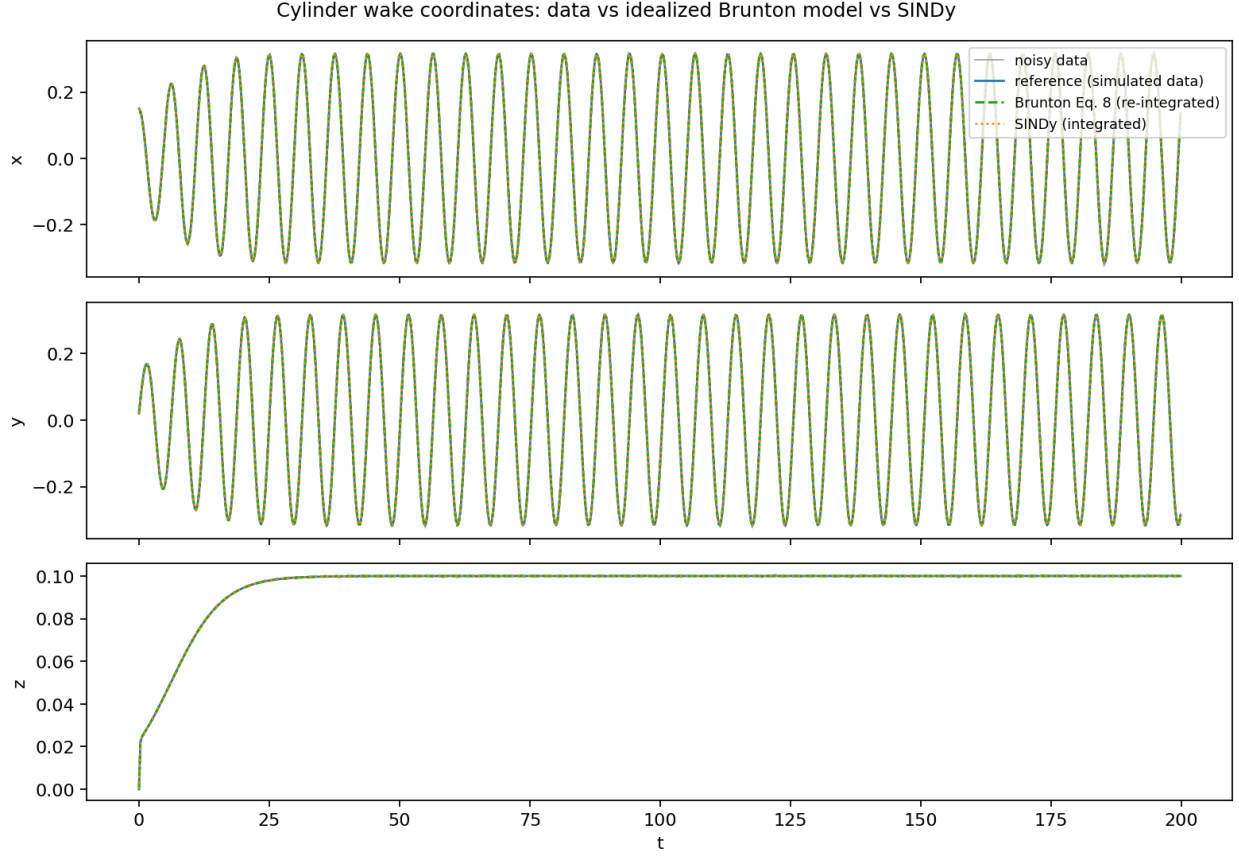
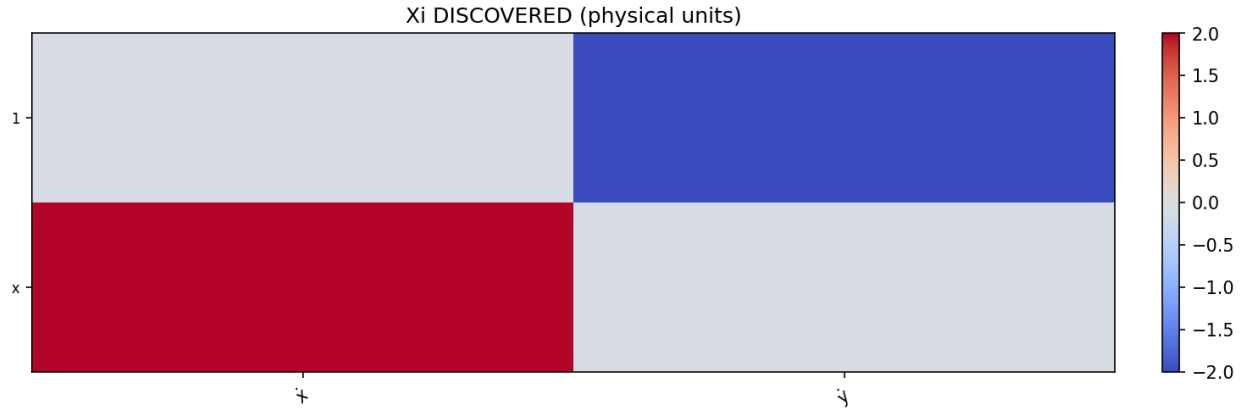


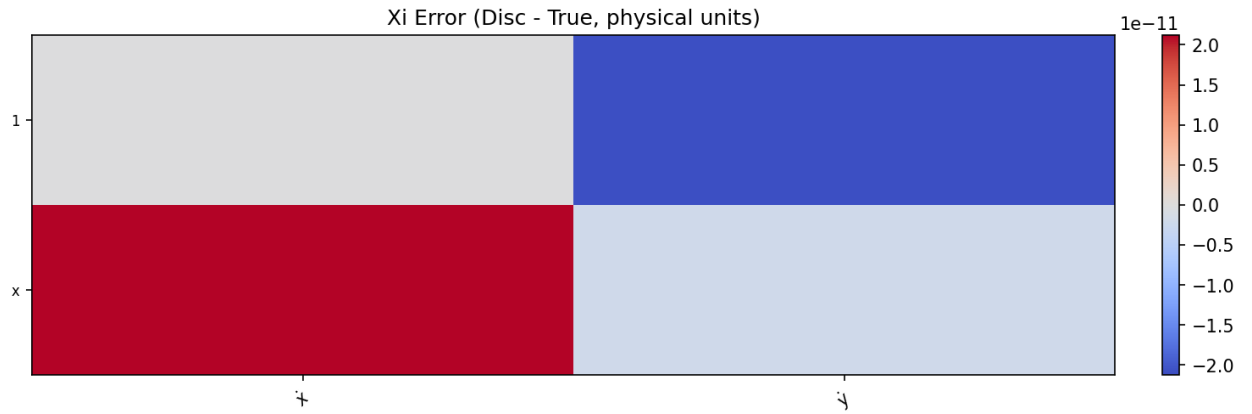
Figure 7: Cylinder wake time series over $T \in [0, 200]$: reference data (blue), direct re-integration of the true Equation (14) (green dashed), and SINDy forward integration (orange dotted). All three agree closely throughout the window, replicating Figure 4 of Brunton et al. (2016).



(a) Ξ_{true}



(b) Ξ_{disc}



(c) Error $\Xi_{\text{disc}} - \Xi_{\text{true}}$

Figure 8: Cylinder wake coefficient heatmaps. The μ , ω , A (bilinear xz , yz), and λ terms of Equation (14) are all recovered correctly.

5 Github Repository

https://github.com/ksommerkohrt/SINDY_Brunton

References

- [1] S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Discovering governing equations from data by sparse identification of nonlinear dynamical systems,” *Proc. Natl. Acad. Sci.*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [2] B. R. Noack, K. Afanasiev, M. Morzyński, G. Tadmor, and F. Thiele, “A hierarchy of low-dimensional models for the transient and post-transient cylinder wake,” *J. Fluid Mech.*, vol. 497, pp. 335–363, 2003.
- [3] H. Zhang et al., “Physics-Informed Machine Learning in Design and Manufacturing: Status and Challenges,” *Journal of Computing and Information Science in Engineering*, vol. 25, no. 4, 2025.
- [4] J. N. Kutz and S. L. Brunton, *Sparse Methods for Data-Driven Modeling and Control*. Cambridge: Cambridge University Press, 2023.
- [5] N. Takeishi and Y. Kawahara, “Sparse Identification of Nonlinear Dynamics With Library Optimization Mechanism: Recursive Long-Term Prediction Perspective,” *IEEE Transactions on Cybernetics*, vol. 56, no. 5, pp. 2475–2488, 2026.
- [6] K. Morales et al., “Sparse identification of nonlinear dynamics and Koopman operators with Shallow Recurrent Decoder Networks,” *Proceedings of the National Academy of Sciences*, vol. 123, no. 19, 2026.
- [7] H. Vaddireddy and A. Rasheed, “SINDy-KANs: Sparse identification of non-linear dynamics through Kolmogorov-Arnold networks,” *arXiv preprint arXiv:2603.18548*, 2026.
- [8] H. Zhang and H. Schaeffer, “On the convergence of the self-adaptive hard thresholding algorithm for sparse identification of nonlinear dynamics,” *SIAM Journal on Applied Dynamical Systems*, vol. 18, no. 2, pp. 715–744, 2019.